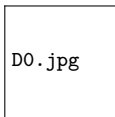


Lecture 10: Relational Algebra

The Mathematical Foundation of Query Languages

Z2004: Database Management Systems



IIT Madras Zanzibar

Monday, 23 March 2026

What is Relational Algebra?

The Query Engine

SQL is a **Declarative** language (you tell the DBMS *what* you want).

Relational Algebra is a **Procedural** language (it defines exactly *how* to compute the result step-by-step). When you write SQL, the database engine translates it into Relational Algebra to execute it!

The Closure Property

Every operation in Relational Algebra takes one or two tables as **input**, and **always produces a new table as output**.

Because the output is a table, we can chain operations together indefinitely!

The 6 Fundamental Operators

σ **Select (Sigma)**: Filter specific rows.

π **Projection (Pi)**: Filter specific columns.

$\cup$ **Union (Cup)**: Combine two sets of rows.

— **Set Difference (Minus)**: Rows in A but not B.

$\times$ **Cartesian Product (Cross)**: Combine everything.

ρ **Rename (Rho)**: Change column/table

The Spreadsheet Analogy

Don't let the Greek letters scare you!

You already know how these operators work if you have ever used Excel or Google Sheets.

Selection (σ)

Filtering Rows

Imagine clicking the "**Filter**" button at the top of a spreadsheet column and unchecking the boxes for data you don't want to see. You are hiding rows based on a condition.

Row 1 (Keep)
Row 2 (Hide)
Row 3 (Keep)

Projection (π)

Filtering Columns

Imagine right-clicking an entire column (like Column B or Column C) in Excel and selecting "**Delete Column**" or "**Hide**". You are selecting which attributes survive.

Col A	Col B	Col C
-------	-------	-------

1. Selection (σ) – Filtering Rows

The σ Operator

Selection extracts **rows** (tuples) that satisfy a specific condition. It filters the table horizontally.

Mathematical Notation:

$$\sigma_{Dept='IT'}(Employee)$$

 **Read as:** “Select rows where Dept is IT from the Employee table.”

SQL Equivalent:

```
SELECT *  
FROM Employee  
WHERE Dept = 'IT';
```

Visualizing Selection

ID	Name	Dept
1	Alice	HR
2	Bob	IT
3	Charlie	IT

 Filter Rows

ID	Name	Dept
2	Bob	IT
3	Charlie	IT

3 & 4. Union ($\cup$) and Set Difference ($-$)

Set Operations

Union ($\cup$): Combines rows from both tables.

Difference ($-$): Rows in A that are NOT in B.

☠ THE UNION TRAP

To combine or subtract tables, they **MUST** be **Union Compatible**:

1. Same number of columns.
2. Matching data types in the exact same order.

Valid vs. Invalid

✓ Valid (Union Compatible)

ID	Name	$\cup$	ID	Name	=	ID	Name
1	Ali		2	Bea		1	Ali
						2	Bea

✗ Invalid (NOT Compatible)

ID	Name	$\cup$	Age	GPA
1	Ali		20	3.8

You cannot stack Names on top of GPAs!

3 & 4. Union ($\cup$) and Set Difference ($-$)

Union ($\cup$): Combines rows from both tables.

```
SELECT ID FROM TableA
UNION
SELECT ID FROM TableB;
```

Difference ($-$): Rows in A that are NOT in B.

```
SELECT ID FROM TableA
EXCEPT
SELECT ID FROM TableB;
```

Visualizing Sets

Table A (ID)

1
2

Table B (ID)

2
3

$A \cup B$

Result

1
2
3

$A - B$

Result

1

5. Cartesian Product ($\times$)

The Data Explosion

Combines **every row** of Table A with **every row** of Table B. If A has 10 rows and B has 10 rows, the output has 100 rows!

Mathematical Notation:

Student $\times$ *Course*

SQL Equivalent:

```
SELECT *  
FROM Student  
CROSS JOIN Course;
```

Usually, we immediately follow a $\times$ with a σ to filter out the nonsense rows. This creates a JOIN!.

Visualizing Cross Join

S_ID	Name
S1	Ali
S2	Bea

$\times$

Course
Math
Art



S_ID	Name	Course
S1	Ali	Math
S1	Ali	Art
S2	Bea	Math
S2	Bea	Art

6. Rename (ρ) and Chaining Operations

Putting It Together

Because every operation outputs a table, we can nest them like math functions. We evaluate from the **inside out**.

Query: "Get the Names of all IT Employees"

$\pi_{Name}(\sigma_{Dept='IT'}(Employee))$

 **Read as:** "Project the Name column from the (Selection of rows where Dept is IT from Employee)."

1. Inner: Find rows where Dept is 'IT' (σ).
2. Outer: Take that result, and grab only the Name column (π).

The Rename Operator (ρ)

Sometimes when chaining, we need to temporarily rename a table or column to avoid confusion.

Notation:

$\rho_{NewName}(Table)$

 **Read as:** "Rename Table as NewName."

SQL Equivalent (AS):

```
SELECT Name AS Employee_Name
FROM Employee AS E
WHERE E.Dept = 'IT';
```

6. Chaining Operations: The Assembly Line

Query: "Get the Names of all IT Employees"

$$\pi_{Name}(\sigma_{Dept='IT'}(Employee))$$

1. Base Table

Alice, HR
Bob, IT
Charlie, IT

Filter Dept='IT'

σ

2. Intermediate

Bob, IT
Charlie, IT

π

Keep Name col

Bob
Charlie

3. Final Output

Because every step outputs a valid table (Closure Property), we evaluate from the inside out!

The Database

M_ID	Title	Genre	Year
101	Dune	Sci-Fi	2021
102	Joker	Drama	2019
103	Avatar 2	Sci-Fi	2022
104	Parasite	Drama	2019



Instructions:

Take 5 minutes and write down the Relational Algebra expressions for the 3 tasks on the right.

The Tasks

Task 1: The Filter

- Write the expression to find all movies released *after* 2020.

Task 2: The Specifics

- Write the expression to get **only the Titles** of all 'Drama' movies.

Task 3: The SQL Translation

- Translate your Relational Algebra answer from **Task 2** into a raw SQL query.

Class Activity: Solutions

Task 1 & Task 2 Answers

Task 1: Movies after 2020 (σ)

$\sigma_{Year > 2020}(Movie)$

Task 2: Titles of Drama movies (π and σ)

$\pi_{Title}(\sigma_{Genre='Drama'}(Movie))$

Task 3: SQL Translation

```
SELECT Title
FROM Movie
WHERE Genre = 'Drama';
```

The Output Tables

Task 1 Result:

M_ID	Title	Genre	Year
101	Dune	Sci-Fi	2021
103	Avatar 2	Sci-Fi	2022

Task 2 Result:

Title
Joker
Parasite

✔ Notice how Task 2 only returns the single column requested by the π operator!

Key Takeaways

- ✓ **Procedural:** Relational Algebra tells the DBMS exactly *how* to get the data.
- ✓ **Selection (σ):** Filters Rows (WHERE).
- ✓ **Projection (π):** Filters Columns (SELECT). Removes duplicates.
- ✓ **Cartesian ($\times$):** Combines every row with every row. Creates a massive table.
- ✓ **Chaining:** Evaluate from the inside out.

Next Lecture (L11)

The Mathematics of JOINS

- ➔ Cartesian product ($\times$) + Selection (σ) = **JOIN ($\bowtie$)**
- ➔ Natural Join vs Theta Join
- ➔ Inner vs Outer Joins